

Major Project:

Online Food Delivery System (Swiggy/Zomato Clone) using MongoDB (NoSQL)

Project Description

The **Online Food Delivery System** is a real-world application where customers can:

- Register and log in
- Browse restaurants
- View menu items
- Place orders
- Make payments
- Track delivery status
- Rate restaurants

Commands:

```
use food_delivery_db
```

```
switched to db food_delivery_db
```

```
db.Customers.insertMany([
```

```
{  customer_id: 1,
```

```
  name: "Saran",
```

```
  email: "saran@gmail.com",
```

```
  phone: "9876543210",
```

```
  city: "Bangalore"},
```

```
{  customer_id: 2,
```

```
  name: "Priya",
```

```
  email: "priya@gmail.com",
```

```
  phone: "9876543211",
```

```
  city: "Hyderabad"},
```

```
{  customer_id: 3,
```

```
  name: "Rahul",
```

```
  email: "rahul@gmail.com",
```

```
    phone: "9876543212",
    city: "Chennai"},
{  customer_id: 4,
  name: "Deepa",
  email: "deepa@gmail.com",
  phone: "9876543213",
  city: "Pune"},
{  customer_id: 5,
  name: "Anjali",
  email: "anjali@gmail.com",
  phone: "9876543214",
  city: "Bangalore"}})
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a341f06777102b36a1824f0'),
    '1': ObjectId('6a341f06777102b36a1824f1'),
    '2': ObjectId('6a341f06777102b36a1824f2'),
    '3': ObjectId('6a341f06777102b36a1824f3'),
    '4': ObjectId('6a341f06777102b36a1824f4') }}
db.createCollection("restaurants")
{ ok: 1 }
db.restaurants.insertMany([
{  restaurant_id: 101,
  restaurant_name: "Pizza Hut",
  city: "Bangalore",
  rating: 4.5},
{  restaurant_id: 102,
  restaurant_name: "Dominos",
```

```
    city: "Hyderabad",
    rating: 4.3},
{ restaurant_id: 103,
  restaurant_name: "Burger King",
  city: "Chennai",
  rating: 4.4},
{ restaurant_id: 104,
  restaurant_name: "KFC",
  city: "Pune",
  rating: 4.2}])
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a341f44777102b36a1824f5'),
    '1': ObjectId('6a341f44777102b36a1824f6'),
    '2': ObjectId('6a341f44777102b36a1824f7'),
    '3': ObjectId('6a341f44777102b36a1824f8') }}
db.createCollection("menu")
{ ok: 1 }
db.menu.insertMany([
{ item_id: 1,
  restaurant_id: 101,
  item_name: "Veg Pizza",
  price: 250},
{ item_id: 2,
  restaurant_id: 101,
  item_name: "Chicken Pizza",
  price: 350},
{ item_id: 3,
```

```

    restaurant_id: 102,
    item_name: "Cheese Burst Pizza",
    price: 300},
{ item_id: 4,
  restaurant_id: 103,
  item_name: "Veg Burger",
  price: 180},
{ item_id: 5,
  restaurant_id: 104,
  item_name: "Chicken Bucket",
  price: 550}]]
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a341f87777102b36a1824f9'),
    '1': ObjectId('6a341f87777102b36a1824fa'),
    '2': ObjectId('6a341f87777102b36a1824fb'),
    '3': ObjectId('6a341f87777102b36a1824fc'),
    '4': ObjectId('6a341f87777102b36a1824fd') }}
db.createCollection("orders")
{ ok: 1 }
db.orders.insertMany([
{ order_id: 1001,
  customer_id: 1,
  restaurant_id: 101,
  items: [ {
    item_name: "Veg Pizza",
    quantity: 2,
    price: 250 }],

```

```

    total_amount: 500,
    status: "Delivered"},
{ order_id: 1002,
  customer_id: 2,
  restaurant_id: 102,
  items: [ {
    item_name: "Cheese Burst Pizza",
    quantity: 1,
    price: 300 } ],
  total_amount: 300,
  status: "Pending"},{
  order_id: 1003,
  customer_id: 3,
  restaurant_id: 103,
  items: [ {
    item_name: "Veg Burger",
    quantity: 2,
    price: 180 } ],
  total_amount: 360,
  status: "Delivered"}})
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a341fa8777102b36a1824fe'),
    '1': ObjectId('6a341fa8777102b36a1824ff'),
    '2': ObjectId('6a341fa8777102b36a182500') }}
db.createCollection("payments")
{ ok: 1 }
db.payments.insertMany([

```

```
    payment_id: 501,  
    order_id: 1001,  
    payment_method: "UPI",  
    amount: 500,  
    status: "Success"},  
{ payment_id: 502,  
  order_id: 1002,  
  payment_method: "Credit Card",  
  amount: 300,  
  status: "Pending"},  
{ payment_id: 503,  
  order_id: 1003,  
  payment_method: "Cash",  
  amount: 360,  
  status: "Success"]])  
{ acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('6a342003777102b36a182501'),  
    '1': ObjectId('6a342003777102b36a182502'),  
    '2': ObjectId('6a342003777102b36a182503') }}  
db.createCollection("delivery_agents")  
{ ok: 1 }  
db.delivery_agents.insertMany([  
{ agent_id: 1,  
  agent_name: "Kumar",  
  city: "Bangalore",  
  rating: 4.8},  
{ agent_id: 2,
```

```
    agent_name: "Arjun",
    city: "Hyderabad",
    rating: 4.6},
{  agent_id: 3,
  agent_name: "Sneha",
  city: "Chennai",
  rating: 4.9}})
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a342022777102b36a182504'),
    '1': ObjectId('6a342022777102b36a182505'),
    '2': ObjectId('6a342022777102b36a182506') }}
db.createCollection("reviews")
{ ok: 1 }
db.reviews.insertMany([
{  review_id: 1,
  customer_id: 1,
  restaurant_id: 101,
  rating: 5,
  comment: "Excellent service"},
{  review_id: 2,
  customer_id: 2,
  restaurant_id: 102,
  rating: 4,
  comment: "Good taste"},
{  review_id: 3,
  customer_id: 3,
  restaurant_id: 103,
```

```
    rating: 5,  
    comment: "Very tasty" ] ]  
{ acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('6a34204a777102b36a182507'),  
    '1': ObjectId('6a34204a777102b36a182508'),  
    '2': ObjectId('6a34204a777102b36a182509') } }
```

Collection 1: Customers

Filter Queries

Task 1

Display all customers.

Commands:

```
db.customers.find()
```

Task 2

Display customers from Bangalore.

Commands:

```
db.customers.find({ city: "Bangalore" })
```

Task 3

Display customers not belonging to Hyderabad.

Commands:

```
db.customers.find({ city: { $ne: "Hyderabad" } })
```

Task 4

Display customers whose names start with 'S'.

Commands:

```
db.customers.find({  
  name: { $regex: "^S" } })
```

Task 5

Display customers whose names end with 'a'.

Commands:

```
db.customers.find({  
  name: { $regex: "a$" }}
```

Task 6

Display customers whose names contain 'ar'.

Commands:

```
db.customers.find({  
  name: { $regex: "ar" }}
```

Task 7

Display customers from Bangalore or Chennai.

Commands:

```
db.customers.find({  
  city: { $in: ["Bangalore", "Chennai"] }}
```

Task 8

Display customers from Bangalore and whose name starts with 'A'.

Commands:

```
db.customers.find({  
  city: "Bangalore",  
  name: { $regex: "^A" }}
```

Task 9

Display customers whose city is in Bangalore, Hyderabad, or Pune.

Commands:

```
db.customers.find({  
  city: { $in: ["Bangalore", "Hyderabad", "Pune"] }}
```

Task 10

Display customers sorted by name.

Commands:

```
db.customers.find().sort({ name: 1 })
```

Aggregation Tasks

Task 11

Count the total number of customers.

Commands:

```
db.customers.aggregate([
  { $count: "TotalCustomers" }])
```

Task 12

Find the number of customers in each city.

Commands:

```
db.customers.aggregate([
  { $group: {
    _id: "$city",
    customerCount: { $sum: 1 } } }])
```

Task 13

Find the city having the maximum number of customers.

Commands:

```
db.customers.aggregate([
  { $group: {
    _id: "$city",
    customerCount: { $sum: 1 } } },
  { $sort: { customerCount: -1 } },
  { $limit: 1 }])
```

Task 14

Display cities having more than one customer.

Commands:

```
db.customers.aggregate([
  { $group: {
    _id: "$city",
```

```
customerCount: { $sum: 1 } }},  
{ $match: { customerCount: { $gt: 1 } } })
```

Task 15

Sort cities by customer count.

Commands:

```
db.customers.aggregate([  
  { $group: {  
    _id: "$city",  
    customerCount: { $sum: 1 } }},  
  { $sort: { customerCount: -1 } }])
```

Collection 2: Restaurants

Filter Queries

Task 16

Display all restaurants.

Commands:

```
db.restaurants.find()
```

Task 17

Display restaurants located in Bangalore.

Commands:

```
db.restaurants.find({  
  city: "Bangalore"})
```

Task 18

Display restaurants having rating greater than 4.3.

Commands:

```
db.restaurants.find({  
  rating: { $gt: 4.3 } })
```

Task 19

Display restaurants having rating between 4.2 and 4.5.

Commands:

```
db.restaurants.find({  
  rating: {  
    $gte: 4.2,  
    $lte: 4.5}}
```

Task 20

Display restaurants whose names start with 'P'.

Commands:

```
db.restaurants.find({  
  restaurant_name: { $regex: "^P" }}
```

Task 21

Display restaurants whose names contain 'King'.

Commands:

```
db.restaurants.find({  
  restaurant_name: { $regex: "King" }}
```

Task 22

Display restaurants located in Chennai or Hyderabad.

Commands:

```
db.restaurants.find({  
  city: { $in: ["Chennai", "Hyderabad"] }}
```

Task 23

Display restaurants not located in Pune.

Commands:

```
db.restaurants.find({  
  city: { $ne: "Pune" }}
```

Task 24

Display restaurants sorted by rating in descending order.

Commands:

```
db.restaurants.find().sort({ rating: -1 })
```

Task 25

Display the top 3 restaurants based on rating.

Commands:

```
db.restaurants.find().sort({ rating: -1 }).limit(3)
```

Aggregation Tasks

Task 26

Find the average restaurant rating.

Commands:

```
db.restaurants.aggregate([
  { $group: {
    _id: null,
    averageRating: { $avg: "$rating" }}}])
```

Task 27

Find the highest rating.

Commands:

```
db.restaurants.aggregate([
  { $group: {
    _id: null,
    highestRating: { $max: "$rating" } }}])
```

Task 28

Find the lowest rating.

Commands:

```
db.restaurants.aggregate([
  { $group: {
    _id: null,
    lowestRating: { $min: "$rating" } }}])
```

Task 29

Count the number of restaurants in each city.

Commands:

```
db.restaurants.aggregate([
  { $group: {
    _id: "$city",
    restaurantCount: { $sum: 1 } } })
```

Task 30

Display cities having more than one restaurant.

Commands:

```
db.restaurants.aggregate([
  { $group: {
    _id: "$city",
    restaurantCount: { $sum: 1 } } },
  { $match: {
    restaurantCount: { $gt: 1 } } })
```

Collection 3: Menu

Filter Queries

Task 31

Display all menu items.

Commands:

```
db.menu.find()
```

Task 32

Display menu items costing more than ₹250.

Commands:

```
db.menu.find({
  price: { $gt: 250 } })
```

Task 33

Display menu items costing between ₹200 and ₹400.

Commands:

```
db.menu.find({  
  price: {  
    $gte: 200,  
    $lte: 400}})
```

Task 34

Display menu items belonging to restaurant ID 101.

Commands:

```
db.menu.find({  
  price: {  
    $gte: 200,  
    $lte: 400  }})
```

Task 35

Display menu items whose names start with 'C'.

Commands:

```
db.menu.find({  
  item_name: { $regex: "^C" }})
```

Task 36

Display menu items whose names contain 'Pizza'.

Commands:

```
db.menu.find({  
  item_name: { $regex: "Pizza" }})
```

Task 37

Display menu items sorted by price in ascending order.

Commands:

```
db.menu.find().sort({ price: 1 })
```

Task 38

Display the top 3 expensive menu items.

Commands:

```
db.menu.find().sort({ price: -1 }).limit(3)
```

Task 39

Display menu items whose prices are less than ₹300.

Commands:

```
db.menu.find({  
  price: { $lt: 300 }}
```

Task 40

Display menu items whose prices are not equal to ₹350.

Commands:

```
db.menu.find({  
  price: { $ne: 350 }}
```

Aggregation Tasks

Task 41

Find the average menu price.

Commands:

```
db.menu.aggregate([  
  
  { $group: {  
    _id: null,  
    averagePrice: { $avg: "$price" }}}])
```

Task 42

Find the maximum menu price.

Commands:

```
db.menu.aggregate([  
  
  { $group: {  
    _id: null,
```



```
maximumPrice: { $max: "$price" } }]))
```

Task 43

Find the minimum menu price.

Commands:

```
db.menu.aggregate([  
  { $group: {  
    _id: null,  
    minimumPrice: { $min: "$price" } } })
```

Task 44

Find the total price of all menu items.

Commands:

```
db.menu.aggregate([  
  { $group: {  
    _id: null,  
    totalPrice: { $sum: "$price" } } })
```

Task 45

Find the number of menu items available in each restaurant.

Commands:

```
db.menu.aggregate([  
  { $group: {  
    _id: "$restaurant_id",  
    itemCount: { $sum: 1 } } })
```

Collection 4: Orders

Filter Queries

Task 46

Display all orders.

Commands:

```
db.orders.find()
```

Task 47

Display delivered orders.

Commands:

```
db.orders.find({  
  status: "Delivered"})
```

Task 48

Display pending orders.

Commands:

```
db.orders.find({  
  status: "Pending"})
```

Task 49

Display orders having total amount greater than ₹400.

Commands:

```
db.orders.find({  
  total_amount: { $gt: 400 }})
```

Task 50

Display orders having total amount between ₹300 and ₹600.

Commands:

```
db.orders.find({  
  total_amount: {  
    $gte: 300,  
    $lte: 600  }})
```

Task 51

Display orders placed by customer ID 1.

Commands:

```
db.orders.find({  
  customer_id: 1})
```

Task 52

Display orders sorted by amount in descending order.

Commands:

```
db.orders.find().sort({ total_amount: -1 })
```

Task 53

Display the top 3 highest orders.

Commands:

```
db.orders.find().sort({ total_amount: -1 }).limit(3)
```

Task 54

Display orders whose status is not "Delivered".

Commands:

```
db.orders.find({
  status: { $ne: "Delivered" }})
```

Task 55

Display orders having amount greater than ₹300 and status "Delivered".

Commands:

```
db.orders.find({
  total_amount: { $gt: 300 },
  status: "Delivered"})
```

Aggregation Tasks

Task 56

Find total revenue generated.

Commands:

```
db.orders.aggregate([
{ $group: {
  _id: null,
  totalRevenue: { $sum: "$total_amount" } }}})
```

Task 57

Find average order amount.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: null,
    averageOrderAmount: { $avg: "$total_amount" } } })
```

Task 58

Find highest order amount.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: null,
    highestOrderAmount: { $max: "$total_amount" } } })
```

Task 59

Find lowest order amount.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: null,
    lowestOrderAmount: { $min: "$total_amount" } } })
```

Task 60

Count orders based on status.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: "$status",
    orderCount: { $sum: 1 } } })
```

Task 61

Display statuses having more than one order.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: "$status",
    orderCount: { $sum: 1 } }},
  { $match: {
    orderCount: { $gt: 1 } }]])
```

Task 62

Display order statuses sorted by order count.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: "$status",
    orderCount: { $sum: 1 } }},
  { $sort: { orderCount: -1 } }]])
```

Task 63

Find the total revenue generated from delivered orders.

Commands:

```
db.orders.aggregate([
  { $match: {
    status: "Delivered" }},
  { $group: {
    _id: null,
    totalRevenue: { $sum: "$total_amount" } } }]])
```

Task 64

Find the average amount of delivered orders.

Commands:

```
db.orders.aggregate([
```

```
{ $match: {  
    status: "Delivered"  }},  
{ $group: {  
    _id: null,  
    averageAmount: { $avg: "$total_amount" }  }}})
```

Task 65

Find the number of orders placed by each customer.

Commands:

```
db.orders.aggregate([  
{ $group: {  
    _id: "$customer_id",  
    orderCount: { $sum: 1 }  }}})
```

Collection 5: Payments

Filter Queries

Task 66

Display all payments.

Commands:

```
db.payments.find()
```

Task 67

Display successful payments.

Commands:

```
db.payments.find(status:"Success")
```

Task 68

Display pending payments.

Commands:

```
db.payments.find({  
    status: "Pending"})
```

Task 69

Display payments made through UPI.

Commands:

```
db.payments.find({  
  payment_method: "UPI"})
```

Task 70

Display payments whose amount is greater than ₹300.

Commands:

```
db.payments.find({  
  amount: { $gt: 300 }})
```

Task 71

Display payments sorted by amount.

Commands:

```
db.payments.find().sort({ amount: -1 })
```

Task 72

Display top 3 payments.

Commands:

```
db.payments.find().sort({ amount: -1 }).limit(3)
```

Task 73

Display payments whose status is not "Success".

Commands:

```
db.payments.find({  
  status: { $ne: "Success" }})
```

Task 74

Display payments made using Cash or UPI.

Commands:

```
db.payments.find({  
  payment_method: {
```

```
$in: ["Cash", "UPI"]}}})
```

Task 75

Display payments whose amount lies between ₹300 and ₹500.

Commands:

```
db.payments.find({  
  amount: {  
    $gte: 300,  
    $lte: 500  })
```

Aggregation Tasks

Task 76

Find the total amount received.

Commands:

```
db.payments.aggregate([  
  { $group: {  
    _id: null,  
    totalAmountReceived: { $sum: "$amount" }  }}})
```

Task 77

Find the average payment amount.

Commands:

```
db.payments.aggregate([  
  { $group: {  
    _id: null,  
    averagePaymentAmount: { $avg: "$amount" }  }}})
```

Task 78

Find the highest payment amount.

Commands:

```
db.payments.aggregate([  
  { $group: {
```



```
_id: null,  
highestPaymentAmount: { $max: "$amount" } }]))
```

Task 79

Find the lowest payment amount.

Commands:

```
db.payments.aggregate([  
  { $group: {  
    _id: null,  
    lowestPaymentAmount: { $min: "$amount" } } }]))
```

Task 80

Count payments by payment method.

Commands:

```
db.payments.aggregate([  
  { $group: {  
    _id: "$payment_method",  
    paymentCount: { $sum: 1 } } }]))
```

Task 81

Count payments by status.

Commands:

```
db.payments.aggregate([  
  { $group: {  
    _id: "$status",  
    paymentCount: { $sum: 1 } } }]))
```

Task 82

Display payment methods having more than one transaction.

Commands:

```
db.payments.aggregate([  
  { $group: {
```

```
    _id: "$payment_method",
    paymentCount: { $sum: 1 }  }},
{ $match: {
    paymentCount: { $gt: 1 }  }}})
```

Task 83

Find the total amount received through UPI.

Commands:

```
db.payments.aggregate([
{ $match: {
    payment_method: "UPI"  }},
{ $group: {
    _id: null,
    totalUPIAmount: { $sum: "$amount" }  }}})
```

Task 84

Find the average amount received through Credit Card.

Commands:

```
db.payments.aggregate([
{ $match: {
    payment_method: "Credit Card"  }},
{ $group: {
    _id: null,
    averageCreditCardAmount: { $avg: "$amount" }  }}})
```

Task 85

Display payment methods sorted by total transaction amount.

Commands:

```
db.payments.aggregate([
{ $match: {
    payment_method: "Credit Card"  }},
```

```
{ $group: {  
  _id: null,  
  averageCreditCardAmount: { $avg: "$amount" }  } })
```

Collection 6: Delivery Agents

Filter Queries

Task 86

Display all delivery agents.

Commands:

```
db.delivery_agents.find()
```

Task 87

Display delivery agents from Bangalore.

Commands:

```
db.delivery_agents.find({  
  city: "Bangalore"})
```

Task 88

Display agents having rating greater than 4.7.

Commands:

```
db.delivery_agents.find({  
  rating: { $gt: 4.7 } })
```

Task 89

Display agents whose names start with 'A'.

Commands:

```
db.delivery_agents.find({  
  agent_name: { $regex: "^A" } })
```

Task 90

Display agents sorted by rating.

Commands:

```
db.delivery_agents.find().sort({ rating: -1 })
```

Aggregation Tasks

Task 91

Find the average agent rating.

Commands:

```
db.delivery_agents.aggregate([
  { $group: {
    _id: null,
    averageRating: { $avg: "$rating" }  }}})
```

Task 92

Find the highest rating.

Commands:

```
db.delivery_agents.aggregate([
  { $group: {
    _id: null,
    highestRating: { $max: "$rating" }  }}})
```

Task 93

Find the lowest rating.

Commands:

```
db.delivery_agents.aggregate([
  { $group: {
    _id: null,
    lowestRating: { $min: "$rating" }  }}})
```

Task 94

Count agents city-wise.

Commands:

```
db.delivery_agents.aggregate([
  { $group: {
    _id: "$city",
```

```
agentCount: { $sum: 1 } }]))
```

Task 95

Display cities having more than one agent.

Commands:

```
db.delivery_agents.aggregate([  
  { $group: {  
    _id: "$city",  
    agentCount: { $sum: 1 } }},  
  { $match: {  
    agentCount: { $gt: 1 } }]])
```

Collection 7: Reviews

Filter Queries

Task 96

Display all reviews.

Commands:

```
db.reviews.find()
```

Task 97

Display reviews with rating 5.

Commands:

```
db.reviews.find({  
  rating: 5})
```

Task 98

Display reviews with rating greater than 4.

Commands:

```
db.reviews.find({  
  rating: { $gt: 4 } })
```

Task 99

Display reviews given by customer ID 1.

Commands:

```
db.reviews.find({
  customer_id: 1})
```

Task 100

Display reviews sorted by rating.

Commands:

```
db.reviews.find().sort({ rating: 1 })
```

Aggregation Tasks**Task 101**

Find the average review rating.

Commands:

```
db.reviews.aggregate([
{ $group: {
  _id: null,
  averageRating: { $avg: "$rating" }  }}})
```

Task 102

Find the highest review rating.

Commands:

```
db.reviews.aggregate([
{ $group: {
  _id: null,
  highestRating: { $max: "$rating" }  }}})
```

Task 103

Find the lowest review rating.

Commands:

```
db.reviews.aggregate([
{ $group: {
  _id: null,
```

```
lowestRating: { $min: "$rating" } }]))
```

Task 104

Count reviews for each restaurant.

Commands:

```
db.reviews.aggregate([
{ $group: {
  _id: "$restaurant_id",
  reviewCount: { $sum: 1 } } ]])
```

Task 105

Display restaurants having more than one review.

Commands:

```
db.reviews.aggregate([
{ $group: {
  _id: "$restaurant_id",
  reviewCount: { $sum: 1 } } },
{ $match: {
  reviewCount: { $gt: 1 } } ]])
```

Task 106

Find the average rating for each restaurant.

Commands:

```
db.reviews.aggregate([
{ $group: {
  _id: "$restaurant_id",
  averageRating: { $avg: "$rating" } } ]])
```

Task 107

Display restaurants sorted by average rating.

Commands:

```
db.reviews.aggregate([
```

```
{ $group: {  
  _id: "$restaurant_id",  
  averageRating: { $avg: "$rating" }  }},  
{ $sort: { averageRating: -1 }}}
```

Task 108

Find the total number of 5-star reviews.

Commands:

```
db.reviews.aggregate([  
{ $match: {  
  rating: 5  }},  
{ $count: "totalFiveStarReviews"}])
```

Task 109

Count reviews based on rating.

Commands:

```
db.reviews.aggregate([  
{ $group: {  
  _id: "$rating",  
  reviewCount: { $sum: 1 }  } }])
```

Task 110

Display ratings having more than one review.

Commands:

```
db.reviews.aggregate([  
{ $group: {  
  _id: "$rating",  
  reviewCount: { $sum: 1 }  } },  
{ $match: {  
  reviewCount: { $gt: 1 }  } }])
```

Advanced Project-Level Aggregation Tasks

Task 111

Find the restaurant generating the highest revenue.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: "$restaurant_id",
    totalRevenue: { $sum: "$total_amount" } }},
  { $sort: { totalRevenue: -1 }},
  { $limit: 1}])
```

Task 112

Find the customer who placed the maximum number of orders.

Commands:

```
db.orders.aggregate([
  { $group: {
    _id: "$customer_id",
    totalOrders: { $sum: 1 } }},
  { $sort: { totalOrders: -1 }},
  { $limit: 1}])
```

Task 113

Find the most popular payment method.

Commands:

Task 114

Find the city with the maximum customers.

Commands:

```
db.payments.aggregate([
  { $group: {
    _id: "$payment_method",
    transactionCount: { $sum: 1 } }},
```

```
{ $sort: { transactionCount: -1 } },  
{ $limit: 1 } ] ] )
```

Task 115

Find the city having the highest number of restaurants.

Commands:

```
db.customers.aggregate([  
  { $group: {  
    _id: "$city",  
    customerCount: { $sum: 1 } } },  
  { $sort: { customerCount: -1 } },  
  { $limit: 1 } ] ] )
```

Task 116

Find the top 3 restaurants based on average ratings.

Commands:

```
db.reviews.aggregate([  
  { $group: {  
    _id: "$restaurant_id",  
    averageRating: { $avg: "$rating" } } },  
  { $sort: { averageRating: -1 } },  
  { $limit: 3 } ] ] )
```

Task 117

Find the top 5 customers based on order count.

Commands:

```
db.reviews.aggregate([  
  { $group: {  
    _id: "$restaurant_id",  
    averageRating: { $avg: "$rating" } } },  
  { $sort: { averageRating: -1 } },
```

```
{ $limit: 3]])
```

Task 118

Find the percentage of delivered orders.

Commands:

```
db.orders.aggregate([  
  { $group: {  
    _id: "$customer_id",  
    orderCount: { $sum: 1 } }},  
  { $sort: { orderCount: -1 }},  
  { $limit: 5}])
```

Task 119

Find the average order amount city-wise.

Commands:

```
db.orders.aggregate([  
  { $lookup: {  
    from: "customers",  
    localField: "customer_id",  
    foreignField: "customer_id",  
    as: "customer" }},  
  { $unwind: "$customer"},  
  { $group: {  
    _id: "$customer.city",  
    averageOrderAmount: { $avg: "$total_amount" } }},  
  { $sort: { averageOrderAmount: -1 }]])
```